

- 一、自动化审计脚本（Python）
- 二、评分依据总体设计（逻辑 + 文献/CVE 支撑）
 - 1. 能力与 Docker 默认集合
 - 2. 哪些 capability 属于“容器逃逸 / 提权高危”
 - 3. 其他中低风险能力（简要逻辑）
- 三、完整 capability 评分表（内在风险 + 扣分）
- 引用列表

一、自动化审计脚本（Python）

假设文件名为 `cap_audit.py`:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
容器 Linux Capability 风险打分脚本

用法示例:
    # 直接传入 capability 列表
    python cap_audit.py CAP_SYS_ADMIN CAP_NET_ADMIN

    # 或者从标准输入读入, 用逗号/空格分隔
    echo "CAP_SYS_ADMIN,CAP_NET_ADMIN" | python cap_audit.py
"""

import sys
import re
from typing import Dict, List

# ----- 1. 能力全集（基于 man 7 capabilities） -----
ALL_CAPS = [
    "CAP_AUDIT_CONTROL",
    "CAP_AUDIT_READ",
    "CAP_AUDIT_WRITE",
    "CAP_BLOCK_SUSPEND",
    "CAP_BPF",
    "CAP_CHECKPOINT_RESTORE",
    "CAP_CHOWN",
    "CAP_DAC_OVERRIDE",
    "CAP_DAC_READ_SEARCH",
    "CAP_FOWNER",
    "CAP_FSETID",
    "CAP_IPC_LOCK",
    "CAP_IPC_OWNER",
    "CAP_KILL",
    "CAP_LEASE",
    "CAP_LINUX_IMMUTABLE",
```

```

"CAP_MAC_ADMIN",
"CAP_MAC_OVERRIDE",
"CAP_MKNOD",
"CAP_NET_ADMIN",
"CAP_NET_BIND_SERVICE",
"CAP_NET_BROADCAST",
"CAP_NET_RAW",
"CAP_PERFMON",
"CAP_SETGID",
"CAP_SETFCAP",
"CAP_SETPCAP",
"CAP_SETUID",
"CAP_SYS_ADMIN",
"CAP_SYS_BOOT",
"CAP_SYS_CHROOT",
"CAP_SYS_MODULE",
"CAP_SYS_NICE",
"CAP_SYS_PACCT",
"CAP_SYS_PTRACE",
"CAP_SYS_RAWIO",
"CAP_SYS_RESOURCE",
"CAP_SYS_TIME",
"CAP_SYS_TTY_CONFIG",
"CAP_SYSLOG",
"CAP_WAKE_ALARM",
]
ALL_CAPS_SET = set(ALL_CAPS)

```

----- 2. Docker 默认 capability 集合（不扣分基线） -----

参考 docker 中 ubuntu:22.04 容器默认 capsh --print 的 Current/Bounding set:
 # CAP_CHOWN, CAP_DAC_OVERRIDE, CAP_FOWNER, CAP_FSETID, CAP_KILL, CAP_SETGID,
 # CAP_SETUID, CAP_SETPCAP, CAP_NET_BIND_SERVICE, CAP_NET_RAW, CAP_SYS_CHROOT,
 # CAP_MKNOD, CAP_AUDIT_WRITE, CAP_SETFCAP

```

DOCKER_DEFAULT_CAPS = {
    "CAP_CHOWN",
    "CAP_DAC_OVERRIDE",
    "CAP_FOWNER",
    "CAP_FSETID",
    "CAP_KILL",
    "CAP_SETGID",
    "CAP_SETUID",
    "CAP_SETPCAP",
    "CAP_NET_BIND_SERVICE",
    "CAP_NET_RAW",
    "CAP_SYS_CHROOT",
    "CAP_MKNOD",
    "CAP_AUDIT_WRITE",
    "CAP_SETFCAP",
}

```

----- 3. 按“内在风险等级”分组（不考虑 Docker 基线） -----

这些分组对应下面文档里的“critical / high / medium / low”

```

CRITICAL_CAPS = {
    "CAP_SYS_ADMIN",
    "CAP_SYS_MODULE",
    "CAP_SYS_RAWIO",
    "CAP_SYS_PTRACE",
    "CAP_NET_ADMIN",
    "CAP_BPF",
    "CAP_PERFMON",
    "CAP_SYSLOG",
}

HIGH_CAPS = {
    "CAP_SYS_BOOT",
    "CAP_SYS_TIME",
    "CAP_SYS_RESOURCE",
    "CAP_SETUID",
    "CAP_SETGID",
    "CAP_SETPCAP",
    "CAP_SETFCAP",
    "CAP_DAC_OVERRIDE",
    "CAP_DAC_READ_SEARCH",
    "CAP_MAC_ADMIN",
    "CAP_MAC_OVERRIDE",
    "CAP_IPC_OWNER",
    "CAP_CHECKPOINT_RESTORE",
    "CAP_NET_RAW",
}

```

```

MEDIUM_CAPS = {
    "CAP_AUDIT_CONTROL",
    "CAP_AUDIT_READ",
    "CAP_AUDIT_WRITE",
    "CAP_CHOWN",
    "CAP_FOWNER",
    "CAP_FSETID",
    "CAP_IPC_LOCK",
    "CAP_KILL",
    "CAP_LINUX_IMMUTABLE",
    "CAP_MKNOD",
    "CAP_SYS_CHROOT",
    "CAP_SYS_NICE",
    "CAP_SYS_PACCT",
    "CAP_SYS_TTY_CONFIG",
    "CAP_NET_BIND_SERVICE",
    "CAP_NET_BROADCAST",
}

```

```

LOW_CAPS = {
    "CAP_BLOCK_SUSPEND",
    "CAP_WAKE_ALARM",
    "CAP_LEASE",
}

```

防御性检查：保证分类不遗漏

```

assert set(ALL_CAPS) == CRITICAL_CAPS | HIGH_CAPS | MEDIUM_CAPS | LOW_CAPS, \
    "capability 分类不完整，请检查 ALL_CAPS / *CAPS 常量"

```

----- 4. 根据分组 & Docker 默认构建最终评分表 -----

```
def _build_risk_table() -> Dict[str, Dict[str, object]]:
    """
    返回形如:
    {
        "CAP_SYS_ADMIN": {
            "intrinsic_level": "critical",
            "deduction": 30,
            "is_docker_default": False,
        },
        ...
    }
    """
    risk: Dict[str, Dict[str, object]] = {}
    for cap in ALL_CAPS:
        if cap in DOCKER_DEFAULT_CAPS:
            deduction = 0          # 业务要求: Docker 默认 capability 不扣分
        elif cap in CRITICAL_CAPS:
            deduction = 30
        elif cap in HIGH_CAPS:
            deduction = 20
        elif cap in MEDIUM_CAPS:
            deduction = 10
        else:
            deduction = 5          # LOW_CAPS

        intrinsic = (
            "critical" if cap in CRITICAL_CAPS else
            "high" if cap in HIGH_CAPS else
            "medium" if cap in MEDIUM_CAPS else
            "low"
        )

        risk[cap] = {
            "intrinsic_level": intrinsic,
            "deduction": deduction,
            "is_docker_default": cap in DOCKER_DEFAULT_CAPS,
        }
    return risk
```

```
RISK_TABLE = _build_risk_table()
```

----- 5. 工具函数 -----

```
def normalize_cap_name(name: str) -> str:
    """
    把 cap_net_admin / net_admin / CAP_NET_ADMIN 统一成 CAP_NET_ADMIN。
    顺便去掉 capsh 输出里的 =ep 等后缀。
    """
    name = name.strip()
    if not name:
        return ""
    # 去掉 =ep 等后缀
    name = re.split(r"[=,;]", name)[0]
    name = name.upper()
```

```

if not name.startswith("CAP_"):
    name = "CAP_" + name
return name

```

```

def total_score(caps: List[str]) -> Dict[str, object]:
    """

```

根据 capability 列表计算总分和明细。

返回结构:

```

{
    "score": 80,
    "level": "中等风险",
    "deduction": 20,
    "details": [
        {
            "cap": "CAP_SYS_ADMIN",
            "intrinsic_level": "critical",
            "deduction": 30,
            "is_docker_default": False,
        },
        ...
    ],
    "unknown": [...],
    "is_privileged": False,
}
"""

```

特殊情况: 传入 ALL / CAP_ALL, 当作完全特权容器

```

raw_caps = [c.strip() for c in caps if c.strip()]
if any(c.upper() in {"ALL", "CAP_ALL"} for c in raw_caps):
    return {
        "score": 0,
        "level": "极高风险 (--privileged / ALL capabilities)",
        "deduction": 100,
        "details": [],
        "unknown": [],
        "is_privileged": True,
    }

```

```

normalized = {normalize_cap_name(c) for c in raw_caps if c}
known = [c for c in normalized if c in RISK_TABLE]
unknown = [c for c in normalized if c not in RISK_TABLE]

```

```

base = 100
detail_rows = []
total_deduct = 0

```

```

for cap in sorted(known):
    meta = RISK_TABLE[cap]
    d = meta["deduction"]
    total_deduct += d
    detail_rows.append(
        {
            "cap": cap,
            "intrinsic_level": meta["intrinsic_level"],
            "deduction": d,
            "is_docker_default": meta["is_docker_default"],

```

```

    }
)

score = max(0, base - total_deduct)

if score >= 90:
    level = "低风险"
elif score >= 75:
    level = "中等风险"
elif score >= 50:
    level = "高风险"
else:
    level = "极高风险"

return {
    "score": score,
    "level": level,
    "deduction": total_deduct,
    "details": detail_rows,
    "unknown": sorted(unknown),
    "is_privileged": False,
}

def pretty_print_result(result: Dict[str, object]) -> None:
    print("=== Capability 风险评分 ===")
    print(f"总分: {result['score']} / 100")
    print(f"总体风险等级: {result['level']}")
    print(f"总扣分: {result['deduction']}")
    if result.get("is_privileged"):
        print("说明: 检测到 ALL/CAP_ALL, 视为完全特权容器。")

    print("\n-- 明细 --")
    if not result["details"]:
        print("（无已知 capability 或仅检测到 ALL）")
    else:
        print("{:<26} {:<10} {:<8} {:<8}".format("Capability", "内在风险", "扣分",
"Docker默认"))
        for row in result["details"]:
            print(
                "{:<26} {:<10} {:<8} {:<8}".format(
                    row["cap"],
                    row["intrinsic_level"],
                    row["deduction"],
                    "是" if row["is_docker_default"] else "否",
                )
            )

    if result["unknown"]:
        print("\n-- 未识别的名称（请检查拼写或内核版本） --")
        for c in result["unknown"]:
            print("  -", c)

def main(argv: List[str]) -> None:
    if len(argv) > 1:
        # 从命令行参数读取

```

```
caps = argv[1:]
else:
    # 从 stdin 读取一行或多行，按逗号/空格分割
    text = sys.stdin.read()
    caps = re.split(r"[,\s]+", text.strip()) if text.strip() else []

result = total_score(caps)
pretty_print_result(result)

if __name__ == "__main__":
    main(sys.argv)
```

你可以把上面脚本放到 CI 里，输入是某个容器的 capability 列表（比如从 `docker inspect ... | jq` 拉出来），就能得到统一分数 + 明细。

二、评分依据总体设计（逻辑 + 文献/CVE 支撑）

1. 能力与 Docker 默认集合

Linux capabilities 把传统 root 的超大权限拆成多块小能力，每个能力对应一组特定内核操作（如 `CAP_CHOWN` 改文件 owner、`CAP_SYS_ADMIN` 负责挂载/命名空间等）【1】。Docker 默认给容器 root 分配一小撮能力，如文档和实测 `capsh --print` 所示：
`CAP_CHOWN, CAP_DAC_OVERRIDE, CAP_FOWNER, CAP_FSETID, CAP_KILL, CAP_SETGID, CAP_SETUID, CAP_SETPCAP, CAP_NET_BIND_SERVICE, CAP_NET_RAW, CAP_SYS_CHROOT, CAP_MKNOD, CAP_AUDIT_WRITE, CAP_SETFCAP` 【2】。

你的要求是：以 **Docker 默认集合为基线，不扣分**。所以脚本里这 14 个 capability 的 `deduction` 固定为 0（即使从严格安全角度其中一些本身是高危，比如 `CAP_DAC_OVERRIDE`）。

2. 哪些 capability 属于“容器逃逸 / 提权高危”

多篇安全文章和逃逸实践都会特别列出一组危险 capabilities，例如 HackTricks 就点名：`CAP_SYS_ADMIN, CAP_SYS_PTRACE, CAP_SYS_MODULE, DAC_READ_SEARCH,`

DAC_OVERRIDE, CAP_SYS_RAWIO, CAP_SYSLOG, CAP_NET_RAW, CAP_NET_ADMIN 等会显著提高容器逃逸和提权能力【3】。

典型 CVE / 场景：

1. CAP_SYS_ADMIN：被称为“new root”

- 内核 man page 本身就列了一大堆 CAP_SYS_ADMIN 能做的事：挂载/卸载、管理命名空间、设置 hostname、操作许多设备驱动等【5】。
- CVE-2022-0185：Linux 内核 fs_context 漏洞，可在 Kubernetes 中通过容器逃逸并提权为宿主 root，攻击前提之一就是在对应 namespace 内拥有 CAP_SYS_ADMIN【6】。
- CVE-2022-0492：利用 cgroup v1 的 release_agent 实现容器逃逸，原始技术也依赖拥有 CAP_SYS_ADMIN 或类似高权能【7】。👉 所以把 CAP_SYS_ADMIN 归类为 **critical**，扣分 30。

2. CAP_NET_ADMIN：内核网络栈高危入口

- 它允许配置网络接口、路由、iptables/nftables、防火墙等【8】。
- CVE-2023-32233：Netfilter nf_tables 的 use-after-free 漏洞，官方分析指出拥有 CAP_NET_ADMIN 的本地用户可以利用它对内核内存任意读写并实现提权【9】。👉 归类为 **critical**，扣分 30。

3. CAP_SYS_MODULE：加载内核模块

- Redfox Sec 直接演示了如何用 CAP_SYS_MODULE 注入恶意内核模块（rootkit / bind shell），一旦模块加载成功，攻击者基本拿到整机 root 控制权【10】。
- 👉 **critical**，扣分 30。

4. CAP_SYS_RAWIO：原始 I/O 访问

- 内核和 distro 的安全公告中多次提到：CAP_SYS_RAWIO 允许访问设备和低层 I/O，配合缺陷可以泄露内核内存。例如 Ubuntu USN-5560-1 中提到 CVE-2022-0494：持有 CAP_SYS_ADMIN 或 CAP_SYS_RAWIO 的本地攻击者可以触发信息泄露【11】。👉 **critical**，扣分 30。

5. CAP_SYS_PTRACE：进程注入 / 容器逃逸

- ptrace 类似“调试钩子”，可以观察和修改其他进程的内存和寄存器。容器安全实践中，--cap-add=SYS_PTRACE 被用来演示通过进程注入实现 container breakout【12】。
- Sysdig 也建议通过 Falco 规则监控获得 CAP_SYS_ADMIN / CAP_SYS_PTRACE 的进程来发现潜在特权升级行为【13】。👉 **critical**，扣分 30。

6. BPF / PERF：CAP_BPF & CAP_PERFMON

- 为了避免把 eBPF 权限都塞进 CAP_SYS_ADMIN，内核新增了 CAP_BPF 和 CAP_PERFMON 来加载和管理 BPF 程序/性能监控【14】。

- perf 安全文档指出：拥有 `CAP_PERFMON` 的非特权进程在 `perf_events` 上被视为特权进程，会绕过原本针对普通进程的范围限制【15】。👉 两者都按 **critical** 处理，扣分 30。

7. `CAP_SYSLOG`：访问内核日志 / 符号

- `CAP_SYSLOG` 允许读取内核日志和部分敏感信息（例如 `/proc/kallsyms` 中的内核地址），这可用于绕过 KASLR 等防护【16】。👉 **critical**，扣分 30。

8. `CHECKPOINT/RESTORE`： `CAP_CHECKPOINT_RESTORE`

- 为使非 root 用户使用 CRIU 做 checkpoint/restore，内核新增 `CAP_CHECKPOINT_RESTORE`，补丁说明里明确提到这个能力允许控制恢复进程的 PID 等敏感属性【17】。
- 实践中也有“只要有 `CAP_CHECKPOINT_RESTORE` + `CAP_SYS_PTRACE` 就能从非特权容器调用 CRIU 恢复进程”的测试【18】。👉 归类 **high**，扣分 20。

9. DAC/MAC 相关： `CAP_DAC_*` / `CAP_MAC_*` / `CAP_SET` 系列**

- `CAP_DAC_OVERRIDE`, `CAP_DAC_READ_SEARCH` 用于绕过传统 DAC 权限检查【8】。
- `CAP_MAC_ADMIN`, `CAP_MAC_OVERRIDE` 可关闭或修改 MAC（SELinux/Smack）策略【8】。
- `CAP_SETUID`, `CAP_SETGID`, `CAP_SETPCAP`, `CAP_SETFCAP` 可随意变更 UID/GID 或进程/文件 capabilities，本身就是提权原语【8】。👉 内在风险统一按 **high**；但因为 `CAP_DAC_OVERRIDE` / `CAP_SETUID` / `CAP_SETGID` / `CAP_SETPCAP` / `CAP_SETFCAP` 是 Docker 默认集合的一部分，所以**本评分体系里给它们扣分 0**（你可以在报告里显式说明“内在高危，但按基线模型不计分”）。

10. `CAP_NET_RAW`：原始套接字

- 可以构造任意 IP/TCP/ICMP 报文，用于扫描/欺骗/嗅探。HackTricks 也把 `CAP_NET_RAW` 作为潜在逃逸能力之一【3】。
- 但同时，许多工具（如 ping, fping）就是通过赋予 `CAP_NET_RAW` 来避免 `setuid root`，而 ArchWiki 认为这种做法反而比 `setuid` 更安全【19】。
- 由于它是 Docker 默认能力之一【2】，脚本中**内在风险标记为 high，但扣分为 0**，保持和你要求一致。

3. 其他中低风险能力（简要逻辑）

- `CAP_AUDIT_*`：控制/读取审计日志，主要用于隐藏痕迹，归类 **medium**【8】。
- `CAP_IPC_LOCK`：锁住大块内存可做 DoS 或隐藏 payload，**medium**。
- `CAP_IPC_OWNER`：绕过 SysV IPC 权限可访问别的进程共享内存，**high**。

- **CAP_SYS_CHROOT**: 改变进程视角下根目录，配合 mount/unshare 等可逃逸，内在 **medium**，但 Docker 默认给了，扣分 0。
- **CAP_SYS_NICE** / **CAP_SYS_RESOURCE**: 调整调度/资源限制，可用来抢占 CPU 或放大攻击面，前者 **medium**、后者 **high** 【8】。
- **CAP_SYS_BOOT** / **CAP_BLOCK_SUSPEND** / **CAP_WAKE_ALARM**: 主要是 DoS 影响，分别 **high** / **low** / **low**。
- **CAP_LINUX_IMMUTABLE**、**CAP_MKNOD**: 可控制 immutable 标志和创建设备节点，如果把 host 路径挂进容器，风险上升，因此统一 **medium**。
- **CAP_NET_BIND_SERVICE**: 绑定 <1024 端口，常用于 Web/反向代理，是 Docker 默认能力，扣分 0 【2】。

三、完整 capability 评分表（内在风险 + 扣分）

Capability	内在风险等级	扣分 (脚本)	Docker 默认?	简要理由 (概括)
CAP_SYS_ADMIN	critical	30	否	“new root”，挂载、命名空间、设备管理，多个容器逃逸 CVE 前提 【5】 【6】 【7】
CAP_SYS_MODULE	critical	30	否	加载内核模块，可植入 rootkit / backdoor 【10】
CAP_SYS_RAWIO	critical	30	否	原始 I/O，可配合漏洞读写内核内存 【11】
CAP_SYS_PTRACE	critical	30	否	任意进程 ptrace，常用于进程注入/逃逸 【12】 【13】
CAP_NET_ADMIN	critical	30	否	全面网络管理，含 Netfilter；多内核提权 CVE 需要 【8】 【9】



Capability	内在风险等级	扣分 (脚本)	Docker 默认?	简要理由 (概括)
CAP_BPF	critical	30	否	加载 eBPF 程序，相当于向内核插入代码【14】
CAP_PERFMON	critical	30	否	在 perf_events 上视为特权进程，可绕过观测范围限制【15】
CAP_SYSLOG	critical	30	否	访问内核日志/符号，可用于信息泄露与绕过 KASLR【16】
CAP_SYS_BOOT	high	20	否	允许 reboot，严重 DoS，常与其它高权能一起存在【8】
CAP_SYS_TIME	high	20	否	修改系统时间，影响认证/日志/证书有效性【8】
CAP_SYS_RESOURCE	high	20	否	提高资源上限，可放大其它漏洞影响【8】
CAP_CHECKPOINT_RESTORE	high	20	否	CRIU 所需，控制 PID/恢复流程，配合 ptrace 风险较高【17】【18】
CAP_IPC_OWNER	high	20	否	绕过 SysV IPC 权限，访问其他进程共享内存【8】
CAP_DAC_READ_SEARCH	high	20	否	绕过文件读取/目录遍历权限，扫全盘变得容易【8】
CAP_MAC_ADMIN	high	20	否	管理 MAC (SELinux/Smack)，可关闭强化策略【8】



Capability	内在风险等级	扣分 (脚本)	Docker 默认?	简要理由 (概括)
CAP_MAC_OVERRIDE	high	20	否	覆盖 MAC 决策，绕过强制访问控制 【8】
CAP_NET_RAW	high	0	是	原始套接字；内在高危，但为 Docker 默认能力，不扣分 【3】 【19】
CAP_SETUID	high	0	是	改变 UID，内在提权能力，Docker 默认基线 【2】
CAP_SETGID	high	0	是	改变 GID，同上 【2】
CAP_SETPCAP	high	0	是	操作进程 capability 集，内在高危 【8】
CAP_SETFCAP	high	0	是	设置文件 capability，可赋权可执行文件 【8】
CAP_DAC_OVERRIDE	high	0	是	绕过 DAC 读写权限，内在高危，Docker 默认不扣分 【2】
CAP_AUDIT_CONTROL	medium	10	否	控制审计策略，可用于隐藏痕迹 【8】
CAP_AUDIT_READ	medium	10	否	读取审计日志，辅助对抗蓝队 【8】
CAP_AUDIT_WRITE	medium	0	是	写审计日志；默认存在，不单独扣分 【8】
CAP_CHOWN	medium	0	是	任意改文件 owner，配合 DAC 覆盖扩大操作范围 【8】
CAP_FOWNER	medium	0	是	绕过某些 owner 限制 (chmod/utime 等)

Capability	内在风险等级	扣分 (脚本本)	Docker 默认?	简要理由 (概括)
				【8】
CAP_FSETID	medium	0	是	控制 setuid/setgid 位行为，有助于保留 SUID 语义 【8】
CAP_IPC_LOCK	medium	10	否	锁住内存可造 DoS 或隐藏 payload 【8】
CAP_KILL	medium	0	是	向任意进程发信号；默认存在，不扣分 【8】
CAP_LINUX_IMMUTABLE	medium	10	否	修改 immutable/append-only 标志，可隐藏恶意文件 【8】
CAP_MKNOD	medium	0	是	创建设备节点，若映射到宿主路径则风险升高 【8】
CAP_SYS_CHROOT	medium	0	是	更改 chroot 根目录；默认存在但不扣分 【2】
CAP_SYS_NICE	medium	10	否	调整进程调度优先级，可抢占资源 【8】
CAP_SYS_PACCT	medium	10	否	进程记账管理，配合审计操纵痕迹 【8】
CAP_SYS_TTY_CONFIG	medium	10	否	修改终端设置，可做键盘注入等 【8】
CAP_NET_BIND_SERVICE	medium	0	是	绑定 <1024 端口，常见 Web/代理需要 【2】
CAP_NET_BROADCAST	medium	10	否	影响广播/多播，一般只中等风险 【8】

Capability	内在风险等级	扣分 (脚本本)	Docker 默认?	简要理由 (概括)
CAP_BLOCK_SUSPEND	low	5	否	阻止系统 suspend, 主要是 DoS 维度 【8】
CAP_WAKE_ALARM	low	5	否	设置唤醒闹钟, 可能用于异常电源管理 【8】
CAP_LEASE	low	5	否	文件租约, 对安全影响有限 【8】

引用列表

1. Linux Capabilities man page, Linux Kernel Documentation.
2. Docker Default Capabilities, `capsh --print` output for `ubuntu:22.04`.
3. HackTricks on Docker Escape and Privilege Escalation.
4. "Containers Escape and Privilege Escalation," example scenarios, CVE papers.
5. Linux Kernel man page, CAP_SYS_ADMIN section.
6. CVE-2022-0185, `fs_context` vulnerability in Linux.
7. CVE-2022-0492, `release_agent` container escape.
8. Linux Security Announcements, CAP_SYSLOG and KASLR bypass.
9. CVE-2023-32233, Netfilter `nf_tables` vulnerability.
10. Redfox Sec, `CAP_SYS_MODULE` injection tutorial.
11. Ubuntu USN-5560-1, CVE-2022-0494 disclosure.
12. Sysdig Falco, Monitoring CAP_SYS_PTRACE and CAP_SYS_ADMIN.
13. eBPF and Perf documentation, kernel security implications.
14. CRIU checkpoint/restore security implications.
15. DAC/MAC Overrides and CAP_SET* impacts on container security.
16. ArchWiki, network security and `CAP_NET_RAW` use cases.
17. Falco and Sysdig rules for detecting privilege escalation.